MIT Open Learning Library
Standard Course Extraction

MITx 8.370.1x — SU1: Quantum and classical computing fundamentals
Populated module/evidence-package record for QAI Enterprise Library extraction

Evidence boundary: MIT course evidence and QAI interpretation are kept separate. Unavailable or unverified fields are marked pending/not supplied. Primitive candidates are not production claims.

1. Course Identity

Course title: Quantum and classical computing fundamentals

MITx / course ID: MITx 8.370.1x

Term / course version: 1T2018 (from the supplied MIT OLL course URL)

MIT Open Learning Library URL: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/

Primary domain: Introduction to quantum computation; classical Boolean logic; introduction to quantum mechanics; quantum measurement

Course purpose: The supplied course introduction states that the course begins with an overview of quantum computation, establishes classical computing foundations based on Boolean logic, then introduces quantum mechanics for computation through states, time evolution, measurement and tensor products, followed by features arising from quantum measurement.

Source material available: Course introduction, SU1 hierarchy, lecture titles, concept questions/screenshots, three solved PS#1 reversible-circuit problems, mathematical hints and solution explanations captured during this extraction.

Extraction status: Complete for the supplied SU1 evidence package; notebooks and uninspected course items remain not supplied/pending.

2. Course Fidelity and Source Structure

- Subunit: SU1: Quantum and classical computing fundamentals.
- Lecture/content items captured: Lectures U1.1: History and development of quantum computation; Lectures U1.2: Classical computation and reversibility.
- Problem set captured: PS#1 - Reversible Circuits.
- Three PS#1 implementation problems explicitly retained as first-class implementation evidence: Reversible two-four-three swap; Controlled-controlled swap; Reversible two-input demultiplexer.
- Concept questions are retained separately from the three PS implementation problems.
- Optional reading supplied with the course introduction: Nielsen and Chuang, Chapter 1.
- MIT terminology is preserved at the course-evidence layer; QAI interpretations appear only in later sections.

3. Course Map

Subunit	Item type	Questions / evidence	Source available	Status
SU1	Lecture U1.1	History/development; concept evidence captured	Yes	Captured
SU1	Lecture U1.2	Classical	Yes	Captured

		computation, reversibility, quantum-state foundations		
SU1	PS#1 - Reversible Circuits	3 solved implementation problems	Yes	Captured
SU1	Concept questions	Multiple CQs captured in screenshots during extraction	Yes	Captured
SU1	Optional reading	Nielsen & Chuang, Chapter 1	Supplied	Reference captured
SU1	Notebook / executable assets	No SU1 notebook supplied in this evidence package	No	Pending

4. Module / Subunit Record

Subunit ID / name: SU1: Quantum and classical computing fundamentals

MIT source URL: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/#block-v1:MITx+8.370.1x+1T2018+type@chapter+block@Week1

Learning objectives: Build foundations in classical Boolean computation and reversibility; introduce quantum states, unitary evolution, measurement and tensor products; connect circuit descriptions with mathematical representations.

Lectures / topics: U1.1 history/development of quantum computation; U1.2 classical computation and reversibility; quantum-state, Bloch-sphere, unitary, measurement and tensor-product concepts appearing in the captured questions.

Problem sets: PS#1 - Reversible Circuits.

Questions: Concept questions plus three solved implementation problems; exact total question count for the complete MIT SU1 is not supplied in this extraction.

Important concepts: Universality; decision problems; Boolean-function cardinality; reversibility; fault tolerance; reversible gates; ancilla; compute-use-uncompute; quantum-state equivalence/global phase; Bloch sphere; unitary matrices; eigenstates/rotation axes; Born-rule probabilities; tensor products; circuit-to-unitary composition.

References: Nielsen and Chuang, Chapter 1 (optional reading supplied in course introduction).

5. Lecture / Content Records

U1.1 — History and development of quantum computation

Type: Lecture

MIT source URL: Supplied course page; exact lecture URL retained in the course capture for U1.1/U1.2.

Question count: Specific lecture question count not supplied.

Concepts covered: History/development of quantum computation; relationship to classical computation.

Key mathematical ideas: Conceptual introduction

Implementation relevance: Provides context for quantum computation and the transition into formal computing foundations.

U1.2 — Classical computation and reversibility

Type: Lecture

MIT source URL: Supplied course page; exact lecture URL retained in the course capture for U1.1/U1.2.

Question count: Multiple concept questions captured; exact lecture total not supplied.

Concepts covered: Classical circuit universality; Boolean functions; decision problems; reversibility; reversible gates; ancilla; quantum-state equivalence; Bloch sphere; unitaries; measurement; tensor products; circuit matrices.

Key mathematical ideas: Classical/reversible and introductory quantum foundations

Implementation relevance: Strong implementation relevance: reversible circuit synthesis, state/operator representations, tensor composition and circuit/operator translation.

6. Problem Set Record

Problem set ID / title: PS#1 - Reversible Circuits

MIT source URL: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/#block-v1:MITx+8.370.1x+1T2018+type@sequential+block@ps1

Question count: Three implementation problems explicitly solved and retained in this extraction.

Original problem available: Yes — problem statements were captured from the supplied screenshots.

Solved questions available: Yes — all three captured solutions were marked correct in the supplied screenshots.

Notebook / code available: No notebook supplied. Solutions were entered as reversible-gate text specifications in the course editor.

Related lecture(s): U1.2: Classical computation and reversibility.

7. Solved Question Extraction

The following records preserve the problem/solution relationship at the level supported by the supplied screenshots and discussion.

PS1-Q1 — Reversible two-four-three swap

Problem statement: Four inputs a,b,c,d; swap b/d when a=0 and c/d when a=1, while leaving a unchanged.

Given information: Classical reversible gates: NOT, Fredkin.

Required result: Four inputs a,b,c,d; swap b/d when a=0 and c/d when a=1, while leaving a unchanged.

Solution / reasoning: Accepted sequence: fredkin(a,c,d); not(a); fredkin(a,b,d); not(a). Use a Fredkin gate for the a=1 branch, invert the control so the second Fredkin implements the complementary a=0 branch, then restore a.

Mathematical derivation: Gate composition under reversible transformations; exact Boolean truth-table derivation was not supplied beyond the accepted circuit.

Algorithm / procedure: Accepted sequence: fredkin(a,c,d); not(a); fredkin(a,b,d); not(a).

Key insight: Conditional reversible composition with control restoration.

Assumptions: Classical reversible gates: NOT, Fredkin.

Complexity: Not explicitly supplied by the course problem; gate count is small and explicit for the accepted construction.

Implementation clues: Reversible circuit composition; gate-level implementation.

Validation / result: Course editor reported 'Correct' for the supplied solution and displayed the corresponding graphical circuit.

PS1-Q2 — Controlled-controlled swap

Problem statement: Inputs a,b,c,d and initialized ancilla e=0; swap c,d only when a=b=1 and restore e=0.

Given information: Toffoli, Fredkin, ancilla.

Required result: Inputs a,b,c,d and initialized ancilla e=0; swap c,d only when a=b=1 and restore e=0.

Solution / reasoning: Accepted sequence: toffoli(a,b,e); fredkin(e,c,d); toffoli(a,b,e). Compute the conjunction into ancilla e, use e as the Fredkin control, then uncompute e.

Mathematical derivation: Gate composition under reversible transformations; exact Boolean truth-table derivation was not supplied beyond the accepted circuit.

Algorithm / procedure: Accepted sequence: toffoli(a,b,e); fredkin(e,c,d); toffoli(a,b,e).

Key insight: Compute → use → uncompute; temporary-resource cleanup.

Assumptions: Toffoli, Fredkin, ancilla.

Complexity: Not explicitly supplied by the course problem; gate count is small and explicit for the accepted construction.

Implementation clues: Ancilla/resource management and reversible control.

Validation / result: Course editor reported 'Correct' for the supplied solution and displayed the corresponding graphical circuit.

PS1-Q3 — Reversible two-input demultiplexer

Problem statement: Inputs a,b with fixed c=0,d=0; produce a four-bit one-hot output where n=2b+a.

Given information: CNOT, Fredkin, NOT; fixed-zero inputs.

Required result: Inputs a,b with fixed c=0,d=0; produce a four-bit one-hot output where n=2b+a.

Solution / reasoning: Accepted sequence: cnot(b,c); cnot(a,b); fredkin(a,c,d); fredkin(c,a,b); not(a). Compose reversible primitives to construct routing/one-hot behavior while preserving reversibility.

Mathematical derivation: Gate composition under reversible transformations; exact Boolean truth-table derivation was not supplied beyond the accepted circuit.

Algorithm / procedure: Accepted sequence: cnot(b,c); cnot(a,b); fredkin(a,c,d); fredkin(c,a,b); not(a).

Key insight: Higher-level function synthesis from primitive reversible gates.

Assumptions: CNOT, Fredkin, NOT; fixed-zero inputs.

Complexity: Not explicitly supplied by the course problem; gate count is small and explicit for the accepted construction.

Implementation clues: Reversible functional synthesis and reversible encoding/routing.

Validation / result: Course editor reported 'Correct' for the supplied solution and displayed the corresponding graphical circuit.

7A. Captured Concept-Question Evidence

ID	Question	Captured result / reasoning	Classification
CQ-01	Universality of classical circuits	Circuit model based on AND/NOT gates; circuits are universal and can simulate Turing machines.	Educational knowledge / classical universality
CQ-02	Faster-than-light communication with entanglement?	The flaw in the historical FLASH proposal is that unknown quantum states cannot be copied.	Educational knowledge / no-cloning implication
CQ-03	Mechanisms for fault-tolerance	Massive redundancy, checkpointing and error correction were identified; code rewriting was not selected.	Educational knowledge / fault tolerance
CQ-04	On computational complexity — decision problems	Decision-problem framing was distinguished from optimization/factor-output questions using Rubik's Cube, packing and factoring examples.	Educational knowledge / complexity foundations
CQ-05	Boolean functions — how many?	An n-input Boolean function has 2^(2^n) possible distinct functions.	Mathematical model / combinatorial growth
CQ-06	Reversible computation in practice	A fundamental limitation of physical reversible implementations is sensitivity to small imperfections causing cascading errors.	Educational knowledge / implementation limitation
CQ-07	Hierarchy of reversible gates	For in-place reversible constructions without garbage or extra ancilla, the captured result was A false and B true for the Fredkin/Toffoli construction claims.	Educational knowledge / reversible universality nuance
CQ-08	Equivalent quantum states	All four displayed states were mathematically identical up to a global phase; global phase does not distinguish physical quantum states.	Mathematical model / global phase
CQ-09	Bloch-sphere state labels	|0> at +z; (|0>+|1>)/sqrt(2) at +x; (|0>+i|1>)/sqrt(2) at +y.	Mathematical model / Bloch sphere
CQ-10	Valid unitary matrices	The first and fourth displayed matrices were valid unitary matrices; unitary condition is U†U=I.	Mathematical model / unitary transformation
CQ-11	Rotation axes of a Hadamard gate	Using the supplied pure-state parameterization and the Hadamard eigenstate, the rotation-axis point is	Mathematical model / eigenstate geometry

		(1/sqrt(2),0,1/sqrt(2)).	
CQ-12	Measurement outcome probabilities	For the selected states, probabilities for outcomes 0 and 2 are equal because their corresponding amplitudes have equal magnitudes.	Mathematical model / Born rule
CQ-13	Tensor product of X and Y	$X \otimes Y$ produces a 4×4 operator from the 2×2 Pauli matrices; the smaller matrix defines a block structure.	Mathematical model / tensor product
CQ-14	Matrix description of a quantum circuit	The complete circuit is represented by a unitary matrix obtained by composing the matrices of the gates in circuit order.	Mathematical model / circuit-to-operator mapping

8. Notebook / Computational Asset Record

Notebook name: Not supplied for SU1.

Course / problem-set relationship: PS#1 solutions were entered directly into the MIT reversible-circuit editor; no IPYNB was supplied.

Original source: MIT OLL course/PS#1 material and screenshots supplied in this conversation.

Language: Course gate-description syntax; not a general-purpose programming language.

Framework / SDK: Not supplied.

Dependencies: Not supplied.

Input: Boolean/reversible circuit inputs and fixed ancilla inputs as defined by each PS problem.

Output: Reversible circuit transformation and graphical circuit rendering.

Algorithm / model: NOT, CNOT, Toffoli and Fredkin gate composition.

Execution environment: MIT OLL course editor, as evidenced by the supplied screenshots.

Result: All three supplied PS problems were shown as Correct.

Validation: Course editor correctness plus graphical circuit rendering.

Reusable pattern: Gate-level reversible function synthesis; compute/use/uncompute; conditional routing.

QAI relationship: Potential reference for a reversible-circuit construction layer, subject to independent implementation and validation.

9. Algorithms and Mathematical Models

Model / algorithm	Mathematical form	Inputs	Outputs	Evidence
Classical Boolean-function space	Number of n-input Boolean functions: $2^{(2^n)}$.	n	Boolean functions	Educational/math evidence
Reversible computation	Reversible transformations	Bit/register state	Reversible transformed state	PS implementation evidence

	preserve enough information to permit inversion; primitive gates include NOT, CNOT, Toffoli and Fredkin.			
Quantum state parameterization	$\vert\psi\rangle = \cos(\theta/2)\vert 0\rangle + e^{(i\phi)} \sin(\theta/2)\vert 1\rangle$ for a pure single-qubit state.	θ, ϕ	Bloch-sphere point	Course hint captured
Unitary evolution	$U^\dagger U = I$; inverse is $U^\dagger$.	State and unitary U	Transformed state	Concept evidence
Bloch-sphere mapping	$(x,y,z) = (\sin\theta \cos\phi, \sin\theta \sin\phi, \cos\theta)$.	θ, ϕ	Bloch vector	Concept/hint evidence
Measurement probability	$P(i)=\vert\alpha_i\vert^2$ for computational-basis amplitude α_i.	State amplitudes	Measurement probabilities	Concept evidence
Tensor product	$A\otimes B$ forms a composite operator; each scalar entry of A scales a full B block.	A, B	Composite operator	Concept evidence
Circuit matrix composition	$U_C = U_m \ldots U_2 U_1$; rightmost operation acts first.	Gate operators	Complete circuit unitary	Concept evidence

10. Extracted Patterns

Pattern	Source evidence	Problem-solving structure	Inputs	Outputs	Control / feedback	Evidence	Potential reuse
Reversible Conditional Composition	PS#1-Q1	Compose controlled operations while preserving/recovering control information.	Bit/register states and reversible gates	Transformed reversible register	Explicit control inversion and restoration	Strong within this PS evidence	Reusable reversible-circuit design pattern; validate independently.
Compute → Use → Uncompute	PS1-Q2	Compute a temporary condition into ancilla; use it; uncompute it to restore clean ancilla.	Inputs + clean ancilla	Desired transformation + restored ancilla	Ancilla is temporary control resource	Strong single-problem evidence	Potential reusable QAI circuit/resource pattern.
Higher-level Function from Primitive Gates	PS1-Q3	Construct a demultiplexer/one-hot function from primitive reversible gates.	Inputs + fixed-zero workspace	Reversible encoded output	Gate composition/routing	Strong single-problem evidence	Candidate for reversible function synthesis.
State ↔ Representation Transformation	CQ-08–CQ-12	Move between state-vector, phase, Bloch-sphere and measurement representations while preserving semantics.	Quantum state	Equivalent representation / observable probabilities	Phase/eigenstate geometry	Moderate multi-question evidence	Useful QAI state-model abstraction.
Unitary Operator ↔ Circuit	CQ-10, CQ-14	Represent gates/circuits as unitary operators	Gate matrices / circuit	Complete unitary matrix	Matrix multiplication and tensor	Moderate evidenc	Candidate QAI intermediate representation.

		and compose them into a complete unitary.			embedding	e	
Hierarchical Tensor Composition / Decomposition	CQ-13 plus subsequent analysis	Tensor products create larger structured operators from smaller blocks; structured composites may be analyzed/decomposed.	Subsystem operators	Composite or factored representation	Block/tensor structure	Concept evidence + QAI inference	High-value future capability; requires independent tensor-network/structured-operator validation.
Scoped Structured Computation	Derived QAI pattern from tensor/block discussion	Use detectable block/tensor structure to scope analysis to relevant subsystems before recomposition.	Structured global representation	Local/partial result and recomposed representation	Structural analysis / scoping	QAI inference, not an MIT claim	Future capability; validate for concrete computational benefit.

11. Primitive Candidates

Reversible Gate Composition

Source evidence: PS#1-Q1/Q2/Q3

Pattern: Compose reversible primitive gates into higher-level transformations.

Purpose: Compose reversible primitive gates into higher-level transformations.

Proposed inputs: Reusable reversible circuit construction

Proposed outputs: Gate/register specification

API concept: Gate sequence, register definitions, optional ancilla

Runtime requirements: Reversible gate library/runtime

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Independent truth-table tests, inverse-circuit tests, additional functions

Maturity: Candidate / not production

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

Quantum State Representation

Source evidence: CQ-08–CQ-12

Pattern: Represent and translate quantum states across vector, phase, Bloch and probability views.

Purpose: Represent and translate quantum states across vector, phase, Bloch and probability views.

Proposed inputs: State vector / parameters

Proposed outputs: Equivalent state representation and observables

API concept: State-model layer

Runtime requirements: Quantum state model + measurement interface

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Validate equivalence and measurement predictions across simulators

Maturity: Candidate / future

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

Unitary State Transformation

Source evidence: CQ-10/CQ-14

Pattern: Apply a valid unitary operator to a quantum state and compose circuit operations.

Purpose: Apply a valid unitary operator to a quantum state and compose circuit operations.

Proposed inputs: State + U

Proposed outputs: Transformed state / composite U

API concept: Operator execution

Runtime requirements: Quantum runtime

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Simulator and independent circuit validation

Maturity: Candidate / future

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

Tensor Composition

Source evidence: CQ-13

Pattern: Construct composite operators/states using tensor products and preserve block structure.

Purpose: Construct composite operators/states using tensor products and preserve block structure.

Proposed inputs: Subsystem operators/states

Proposed outputs: Composite operator/state

API concept: Composition/decomposition layer

Runtime requirements: Quantum state/operator model

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Dimension checks, algebraic equivalence and multi-qubit validation

Maturity: Candidate / future

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

Circuit–Operator–Matrix Translation

Source evidence: CQ-14 and related unitary/tensor evidence

Pattern: Translate between equivalent circuit, operator and matrix representations.

Purpose: Translate between equivalent circuit, operator and matrix representations.

Proposed inputs: Circuit/gates/operator

Proposed outputs: Equivalent representation

API concept: Representation/IR layer

Runtime requirements: QAI language/runtime compiler layer

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Round-trip equivalence tests

Maturity: Candidate / future

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

Structured Tensor Scoping

Source evidence: QAI inference from CQ-13 and tensor discussion

Pattern: Analyze structured composite representations and scope computation to relevant blocks/subsystems.

Purpose: Analyze structured composite representations and scope computation to relevant blocks/subsystems.

Proposed inputs: Structured operator/state

Proposed outputs: Scoped representation/results

API concept: Structure analyzer / scope manager

Runtime requirements: QAI Runtime / structured-computation layer

Dependencies: Quantum state/operator model and reversible/quantum gate definitions as applicable.

Validation required: Benchmark against full-matrix methods and structured tensor methods

Maturity: Future capability

Promotion recommendation: Do not promote from SU1 alone; require repeated evidence, independent implementation and validation, consistent with the extraction process.

12. QAI Architecture Mapping

QAI layer	Mapping
QAI Language	Represent Boolean/reversible functions, quantum states,

	gates, circuits, operators and tensor compositions as explicit semantic constructs.
QAI Primitives	Observe, evolve/transform, measure, compose, decompose, route and reversible-gate composition are candidate abstractions; exact canonical primitive status remains pending.
QAI OS	Provide representation, capability and resource abstractions for reversible and quantum computations; SU1 is foundational evidence rather than a complete OS design.
QAI Runtime	Execute reversible/quantum circuits and operator transformations; support gate composition and unitary application.
Hybrid Runtime	Potentially coordinate classical control/analysis with quantum state/operator execution; not directly implemented in SU1.
QAI Control Plane	Potentially manage circuit construction, resource/ancilla constraints and execution policies; compute/use/uncompute is a relevant control pattern.
Quantum Control Plane	Potential future control of quantum operations, state preparation, measurement and structured execution; not directly implemented in SU1.
Adaptive Network Fabric	Not directly evidenced by SU1; future mapping only.
Resource Registry	Relevant to tracking ancilla/workspace and gate/operator resources; future implementation.
Capability Registry	Candidate capabilities: reversible circuit synthesis, unitary transformation, tensor composition, representation translation.
QAI Labs	Natural validation location for reversible circuits, unitary matrices, tensor operators and state/measurement equivalence.
QAI Product Foundry	Potential source of reusable quantum/reversible computational components after validation; not a direct SU1 deliverable.

13. Classification

- Educational knowledge — classical universality, decision-problem framing, fault tolerance, reversible computing and introductory quantum concepts.
- Mathematical model — Boolean-function cardinality, state parameterization, Bloch sphere, unitarity, measurement probability and tensor products.
- Algorithm / procedure — reversible gate constructions in PS#1.
- Pattern — reversible conditional composition; compute/use/uncompute; circuit/operator/matrix representation; hierarchical tensor composition.
- Implementation evidence — three PS#1 reversible circuits executed/accepted by the MIT OLL editor.
- Reusable capability — candidate only; requires independent implementation and validation.
- Future capability — structured tensor scoping/decomposition and broader representation-transformation services.
- Research — not claimed from SU1 alone.

14. Evidence and Provenance

MIT OLL source URL(s): Course: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/ ; SU1 Week 1: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/#block-v1:MITx+8.370.1x+1T2018+type@chapter+block@Week1 ; PS#1: https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.1x+1T2018/course/#block-v1:MITx+8.370.1x+1T2018+type@sequential+block@ps1

Original document / problem-set reference: MITx 8.370.1x, SU1, Lectures U1.1/U1.2, PS#1 - Reversible Circuits; concept-question screenshots supplied in this conversation.

Notebook reference: Not supplied.

Solution reference: Supplied screenshots and reasoning captured during the SU1 extraction session.

External reference: Nielsen and Chuang, Chapter 1, as listed as optional reading in the supplied course introduction.

Evidence limitations: The extraction is based on the supplied course structure, screenshots, questions, hints and solved PS items; it is not a complete independent crawl of every MIT OLL page. Exact total question counts and notebook inventory are therefore not asserted.

15. Cross-Course Convergence

Course / topic	Common pattern	Primitive candidate	Evidence status
MIT 8.371.3x — prior extracted material	Quantum operator → probe → measurement → estimation → feedback	AQSE (from separate course evidence)	Related concept only; not a SU1 primitive
Future MIT quantum courses	State/operator/tensor/reversible foundations may recur	State, unitary, tensor, composition candidates	Pending future course extraction
SU1 internal recurrence	Composition, representation, reversibility appear repeatedly	Reversible composition; representation translation	Supported within SU1

Cross-course convergence is intentionally not treated as validated until the other course evidence is extracted and compared.

16. Reuse and Implementation Assessment

- Direct reuse: MIT gate-description examples can inform reference test cases, but should not be treated as production code.
- Pattern reuse: strong opportunity for reversible composition, compute/use/uncompute, circuit/operator translation and tensor composition.
- Design reference: strong for QAI representation and execution-layer architecture.
- Research input: tensor/block decomposition and structured scoping are promising research/design directions, but require independent evidence.
- Architecture input: high — particularly separation of semantic operation, circuit, operator, matrix and structured representation.
- Prototype candidate: reversible-circuit library and circuit/operator round-trip translator.
- New implementation required: QAI-native APIs, validation harness, runtime integration, structured representation analysis and telemetry.
- Historical / educational reference only: course-history content unless reused for education/knowledge assets.

17. Validation Plan

Reference implementation needed: Yes — implement the three PS#1 circuits plus representative reversible/quantum state and operator examples in the QAI lab.

Simulator validation: Validate reversible truth tables, inverse circuits, unitary conditions, Bloch-state equivalence, measurement probabilities and tensor products using an appropriate quantum simulator.

Hardware / QPU validation: Optional later stage for quantum-state/unitary/measurement primitives; not required to validate classical reversible PS circuits.

Independent problem validation: Construct additional reversible multiplexers, adders, controlled swaps and reversible Boolean functions; test state/operator representation conversions on independent examples.

Cross-course validation: Compare with later MIT courses covering quantum gates, algorithms, tensor networks, estimation, QEC or related operator/state models.

Benchmark: Compare full-matrix versus structured/tensor representations where applicable; measure correctness, memory footprint, execution time and scalability.

Acceptance criteria: Exact functional equivalence; reversibility/invertibility where required; unitary condition $U^\dagger U=I$ for unitary candidates; tensor dimensions and algebraic equivalence; round-trip circuit/operator/matrix consistency; clean ancilla restoration for compute/use/uncompute patterns.

Promotion gate: No production primitive promotion from SU1 alone. Follow Candidate → Specification → Reference Implementation → Validation → Reusable Primitive → Enterprise Library.

18. Final Course Summary

Major concepts: Classical Boolean universality; decision problems; complexity foundations; fault tolerance; reversible computation; reversible gate universality; quantum state equivalence/global phase; Bloch sphere; unitary transformations; measurement probabilities; tensor products; circuit matrix representation.

Important algorithms: Reversible gate-level constructions demonstrated by PS#1; matrix composition for circuit representation; Bloch/state conversion formulas; tensor-product construction.

Important problem-solving patterns: Conditional reversible composition; compute/use/uncompute; higher-level function synthesis from primitive gates; representation equivalence; circuit ↔ operator ↔ matrix; hierarchical tensor composition/decomposition; scoped structured computation as a QAI-derived future pattern.

Important solved questions: All three PS#1 implementation problems; captured concept questions on universality, complexity, reversibility, quantum-state equivalence, Bloch sphere, unitarity, measurement and tensor products.

Important notebooks: None supplied for SU1.

Primitive candidates: Reversible Gate Composition; Quantum State Representation; Unitary State Transformation; Tensor Composition; Circuit–Operator–Matrix Translation; Structured Tensor Scoping.

Enterprise Library contributions: Foundational candidate abstractions for reversible computation, quantum state/operator representation, composition/decomposition and structured computation.

QAI architecture contributions: Strong evidence for a layered representation model and for composition/decomposition as an architectural capability; possible scoped computation over structured tensor/block representations.

Implementation opportunities: QAI reversible circuit library; quantum state model; unitary/operator model; tensor composition service; circuit/operator/matrix IR; structured operator analyzer; QAI lab validation suite.

Evidence gaps: Complete MIT OLL page-by-page inventory, exact question counts, lecture transcript/content details, executable notebooks and independent implementation evidence are not supplied in this extraction.

19. Extraction Status

Area	Status	Notes
Course structure	Complete	SU1 hierarchy captured from supplied course structure; full independent crawl not performed.

Modules / subunits	Complete	SU1 captured.
Lectures	Captured	U1.1 and U1.2 identified; exact full lecture-content inventory pending.
Problem sets	Complete for supplied evidence	PS#1 and three solved problems captured.
Solved questions	Complete for supplied evidence	Three PS problems plus captured CQs summarized.
Notebooks	Pending	No SU1 notebook supplied.
Algorithms	Captured	Reversible circuit constructions and representation procedures.
Mathematical models	Captured	Boolean functions, state/Bloch, unitarity, measurement, tensor product, circuit matrices.
Patterns	Captured	Recurring patterns identified; QAI-derived patterns separated from MIT evidence.
Primitive candidates	Candidate	No production promotion.
QAI mapping	Captured	Architecture mappings recorded.
Cross-course mapping	Pending	Future courses must be extracted before convergence is validated.
Validation	Planned	Reference implementation and independent validation required.
Final review	Complete	SU1 evidence package consolidated.

20. Extraction Rules

- Preserve the actual MIT OLL hierarchy and terminology.
- Keep original problem statements and solved questions associated.
- Treat solved questions as first-class evidence.
- Treat notebooks as implementation evidence when supplied.
- Do not infer implementation maturity from educational material.
- Separate MIT source evidence from QAI interpretation.
- Promote patterns to primitive candidates only when supported by evidence.
- Promote primitives toward the Enterprise Library only after appropriate validation.
- Use cross-course recurrence to strengthen, not replace, validation.
- Master working rule: PRESERVE → EXTRACT → ANALYZE → GENERALIZE → VALIDATE → MAP → PROMOTE.

QAI Retention Snapshot

SU1 should be retained as a foundational evidence package for the QAI stack. Its strongest implementation value is not a single quantum algorithm, but the reusable relationship between primitives, reversible composition, state/operator representations, tensor composition, circuit-to-unitary translation, and structured decomposition. The tensor/block discussion also motivates a future structured-computation capability, but that capability remains a QAI architectural hypothesis until independently validated.

End of SU1 extraction — MITx 8.370.1x